

Delivery Cloud

Plataforma de delivery basada en microservicios, con analítica sobre un data lake, desplegada en AWS

Informe técnico · Proyecto Parcial (Semanas 3 a 6)

Repositorio público (GitHub)	https://github.com/miguelespinozaa-ship-it/ProeyctParcialProp
Aplicación web (AWS Amplify)	https://main.duqtfda5tvcw.amplifyapp.com
API pública (API Gateway, HTTPS)	https://jt3z0elcud.execute-api.us-east-1.amazonaws.com
Fecha	21 de septiembre de 2026

1. Resumen

Delivery Cloud es una plataforma de pedidos de comida con tres roles: **customer** (explora restaurantes, arma su carrito, hace pedidos, los sigue y deja reseñas), **admin** de restaurante (gestiona su menú, ve los platos de cada pedido y lo despacha) y **delivery** (toma pedidos disponibles y los entrega). El backend se compone de **cinco microservicios en Docker**, escritos en tres lenguajes y con tres motores de base de datos distintos. Un sexto flujo, de **Data Science**, extrae el 100 % de las tablas hacia un data lake en S3, las cataloga con AWS Glue y las consulta con Amazon Athena; el microservicio analítico (MS5) expone esas consultas a la aplicación web.

Todo está desplegado en AWS: dos máquinas virtuales de aplicación detrás de un **balanceador interno**, una máquina virtual de base de datos privada, una máquina de ingesta, **API Gateway** como puerta de entrada HTTPS (con VPC Link hacia el balanceador) y la SPA en **AWS Amplify**.

Microservicios	Lenguajes / motores	Registros cargados	Máquinas virtuales	Consultas Athena
5 en Docker	Python, Java, Node.js · MySQL, PostgreSQL, MongoDB	más de 20,000 por base de datos	4 EC2 (2 de App balanceadas)	4 con JOIN y 2 vistas

Cumplimiento de los requisitos

Requisito	Cómo se cumple	Sección
5 microservicios en Docker; 3 con su propia BD, 3 lenguajes, 3 BD (2 SQL y 1 NoSQL)	MS1 FastAPI + MySQL, MS2 Spring Boot + MongoDB, MS3 Express + PostgreSQL	3
Diagrama E/R por BD SQL y JSON de la BD NoSQL; mínimo 2 tablas relacionadas	E/R de MySQL (usuarios 1:N direcciones) y PostgreSQL (orders 1:N order_items); estructura JSON de restaurantes	4
Un microservicio consume a otro	MS3 consulta a MS2 (validar plato y precio) y a MS1 (nombres de clientes)	3.2
Un microservicio sin BD que solo consume a otros	MS4 Agregador/Rastreo: tracking y 3 dashboards con datos de MS1, MS2 y MS3	3
Un microservicio analítico con queries en Athena	MS5 ejecuta consultas con boto3 sobre las vistas del data lake	7
Carga masiva única, mínimo 20,000 registros por BD	seed.py (usuarios), seed.js (pedidos) y seed.js en mongosh (restaurantes)	4.4
2 VMs de producción con balanceador privado, BD en una tercera VM privada, APIs públicas con HTTPS en API Gateway	2 EC2 de App detrás de un ALB interno; API Gateway → VPC Link → ALB; BD en EC2 sin puertos abiertos a internet	6
Documentar las 5 APIs en Swagger UI	Swagger UI accesible en los 5 servicios, incluso a través del API Gateway	6.5
Aplicación web que consuma los 5 servicios (≥ 2 métodos REST por servicio) desplegada en Amplify	SPA React: 5, 9, 8, 4 y 2 llamadas REST a MS1 ... MS5; desplegada en AWS Amplify	5
MV de ingesta, bucket S3 y 3 contenedores Python de ingesta (pull del 100 %)	PP-Ingest-VM con 3 contenedores; S3 con snapshot completo por tabla	7.1
Catálogo en Glue y diagrama E/R que relacione las tablas del catálogo	Crawler sobre S3, 5 tablas; diagrama E/R del catálogo	7.2
4 consultas SQL con JOIN en Athena y 2 vistas	Ejecutadas en Athena con evidencia (ID de ejecución y resultados)	7.3

Requisito	Cómo se cumple	Sección
Diagrama de arquitectura en draw.io con todos los servicios de AWS	infra/diagramas/proyecto-delivery-cloud.drawio (5 páginas)	2
Repositorios públicos en GitHub	Un monorepo público con todo el código fuente	Anexo A

2. Arquitectura de la solución

Diagrama de Arquitectura de Solución — Delivery Cloud (AWS · us-east-1)

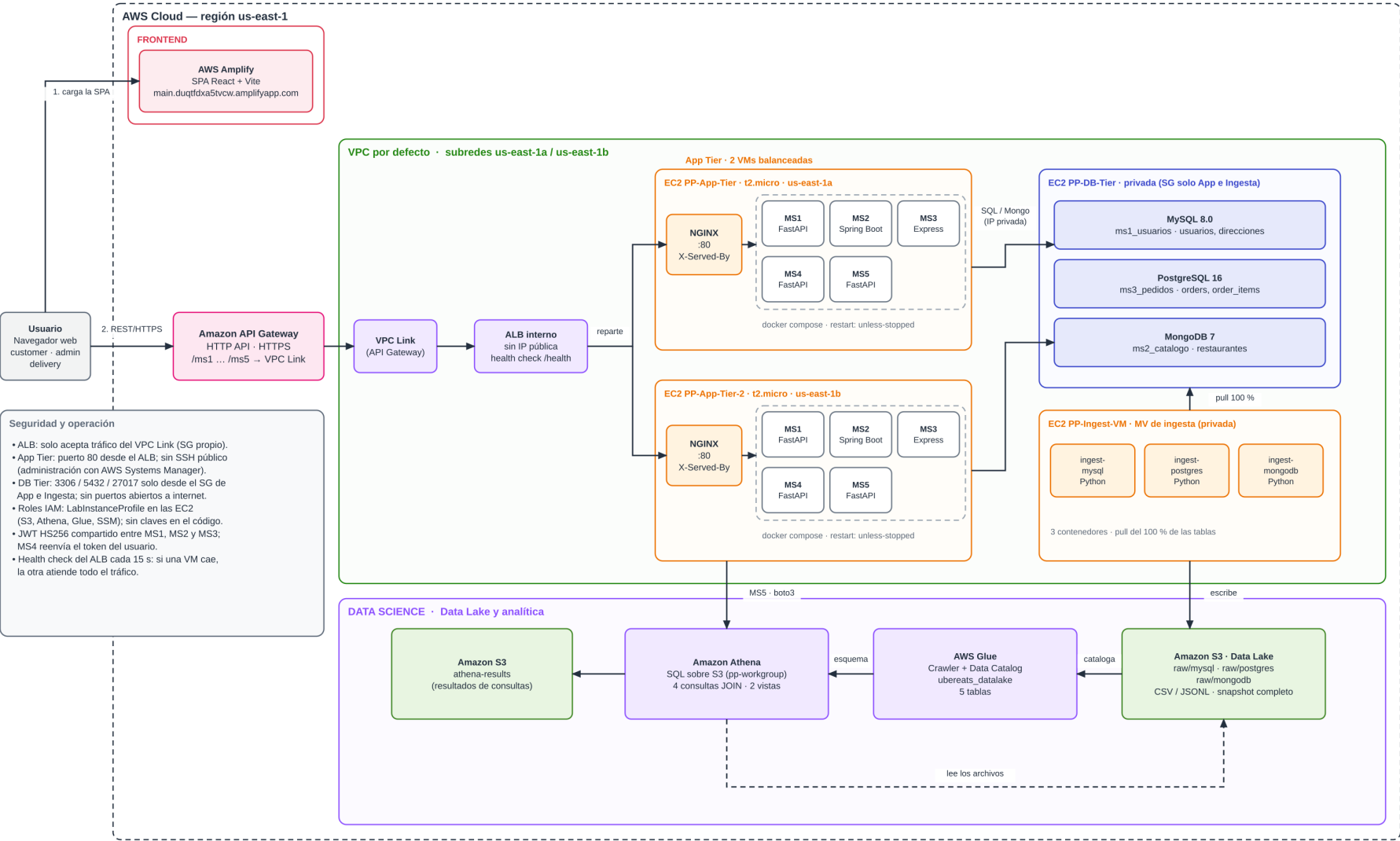


Figura 1. Arquitectura de solución en AWS (editable en `infra/diagramas/proyecto-delivery-cloud.drawio`).

Flujo de una petición

- **1. Carga de la aplicación.** El navegador descarga la SPA React desde AWS Amplify por HTTPS.
- **2. Llamada a la API.** La SPA llama al API Gateway (HTTP API, HTTPS) con rutas /ms1 ... /ms5 y el token JWT del usuario.
- **3. Entrada a la red privada.** API Gateway entra a la VPC por un **VPC Link** hacia el **ALB interno**, que no tiene IP pública y solo acepta tráfico del VPC Link.
- **4. Balanceo.** El ALB reparte entre las dos VMs de App (zonas us-east-1a y 1b) y las saca de rotación si su health check /health falla.
- **5. Enrutamiento.** NGINX de cada VM enruta por prefijo de ruta al microservicio correspondiente y añade la cabecera *X-Served-By* con el nombre de la VM.
- **6. Datos.** Cada microservicio se conecta por IP privada a su base de datos en la VM de BD; MS5 consulta Athena con boto3 usando el rol de la instancia.

Decisiones de diseño

- **Base de datos por servicio.** Cada microservicio es dueño de su esquema; las relaciones entre servicios (customer_id, restaurant_id) son referencias lógicas y se resuelven por API, no por claves foráneas.
- **JWT compartido.** MS1 emite tokens HS256; MS2 y MS3 los validan con el mismo secreto. MS4 reenvía el token del usuario a los servicios que consulta.
- **Servicios sin estado.** Toda la persistencia está en la VM de BD, por eso las dos VMs de App son idénticas y pueden balancearse sin coordinación.
- **Paginado opcional.** Los listados grandes aceptan ?page= y ?page_size=; sin esos parámetros devuelven todo, como antes (sección 8).

3. Microservicios

Servicio	Tecnología	Base de datos	Responsabilidad	Operaciones (Swagger)
MS1 Usuarios / Auth	Python · FastAPI	MySQL 8.0 (usuarios, direcciones)	Registro, login (JWT), perfil, direcciones y consulta de usuarios	12
MS2 Catálogo	Java 17 · Spring Boot	MongoDB 7 (restaurantes con platos y reseñas embebidos)	Restaurantes, menú (CRUD del admin) y reseñas	13
MS3 Pedidos	Node.js · Express	PostgreSQL 16 (orders, order_items)	Crear pedido, cambiar de estado, tomar y entregar, resumen por estado	9
MS4 Agregador / Rastreo	Python · FastAPI	Sin base de datos	Tracking de un pedido y dashboards por rol combinando MS1, MS2 y MS3	5
MS5 Analítico	Python · FastAPI + boto3	Amazon Athena (data lake)	Top de restaurantes, métricas de usuarios y resumen de ventas	5

3.1 Estados y reglas de negocio

- **Pedido:** PEDIDO → ENVIADO (solo el admin dueño del restaurante) → ENTREGADO (solo el repartidor que lo tomó). Un pedido ENVIADO sin repartidor está disponible para que un delivery lo tome (*claim*); si dos lo intentan a la vez, solo uno lo obtiene.
- **Roles:** el registro público no permite crear admins. Cada admin está asociado a un restaurante (restaurant_id en el token) y solo puede tocar su propio menú y sus pedidos.

- **Reseñas:** puntuación de 1 a 5 y comentario obligatorio; al guardar una reseña se recalcula la calificación promedio del restaurante.

3.2 Comunicación entre microservicios

Origen	Destino	Para qué
MS3 Pedidos	MS2 Catálogo	Validar que el plato existe y obtener su precio antes de guardar el pedido
MS3 Pedidos	MS1 Usuarios	Resolver nombres de los clientes de un restaurante (en bloques de 200 ids)
MS4 Agregador	MS1, MS2 y MS3	Componer el tracking del pedido y los dashboards de customer, delivery y admin; si un servicio falla responde con datos parciales o 502 controlado
MS5 Analítico	Amazon Athena	Consultas SQL sobre las vistas v_resumen_ventas_restaurante y v_metricas_usuarios

4. Modelo de datos

Cada base de datos SQL tiene dos tablas relacionadas y se documenta con su diagrama Entidad-Relación; la base NoSQL se documenta con la estructura JSON de su documento.

4.1 MySQL — MS1 (usuarios 1 : N direcciones)

Diagrama Entidad-Relación — MS1 Usuarios/Auth · MySQL 8.0 · base ms1_usuarios

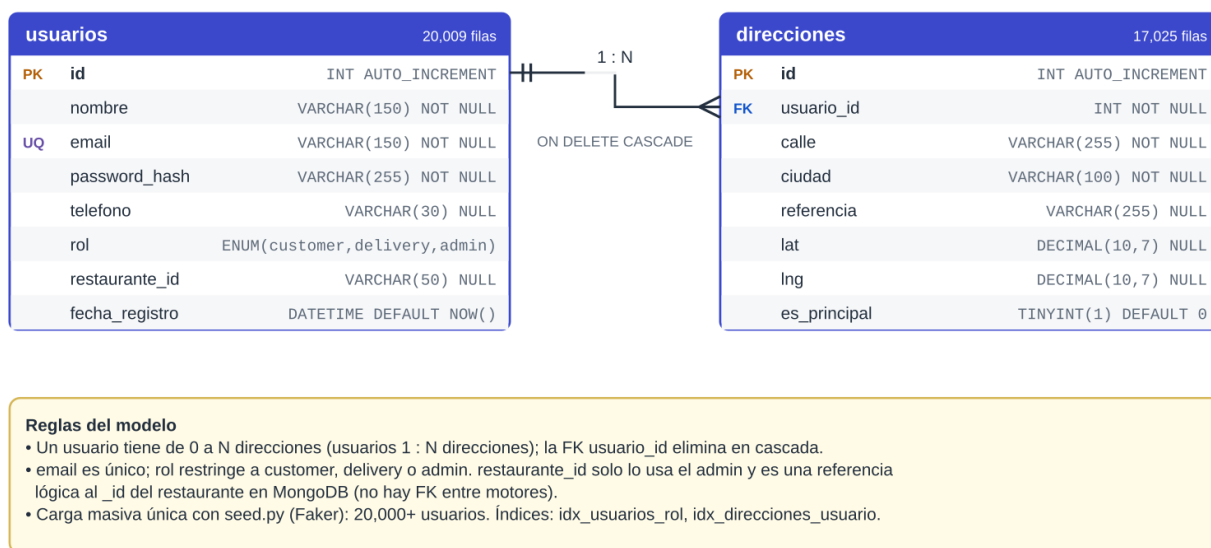


Figura 2. Diagrama E/R de MS1.

4.2 PostgreSQL — MS3 (orders 1 : N order_items)

Diagrama Entidad-Relación — MS3 Pedidos · PostgreSQL 16 · base ms3_pedidos

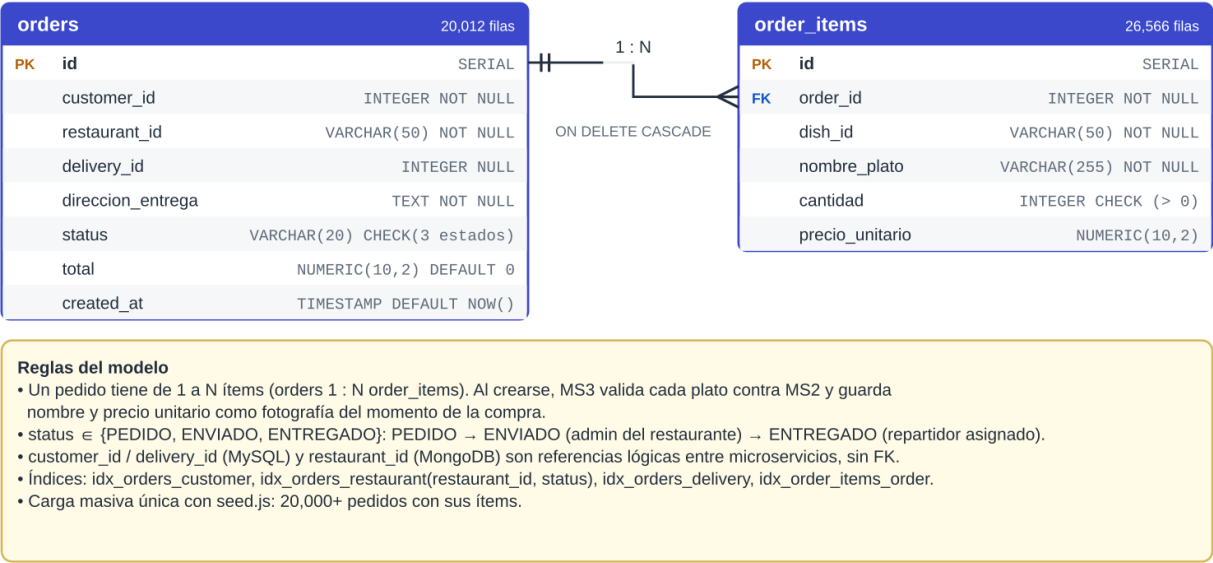


Figura 3. Diagrama E/R de MS3.

4.3 MongoDB — MS2 (restaurantes con platos y reseñas embebidos)

Estructura JSON — MS2 Catálogo de Restaurantes · MongoDB 7 · colección ms2_catalogo.restaurantes

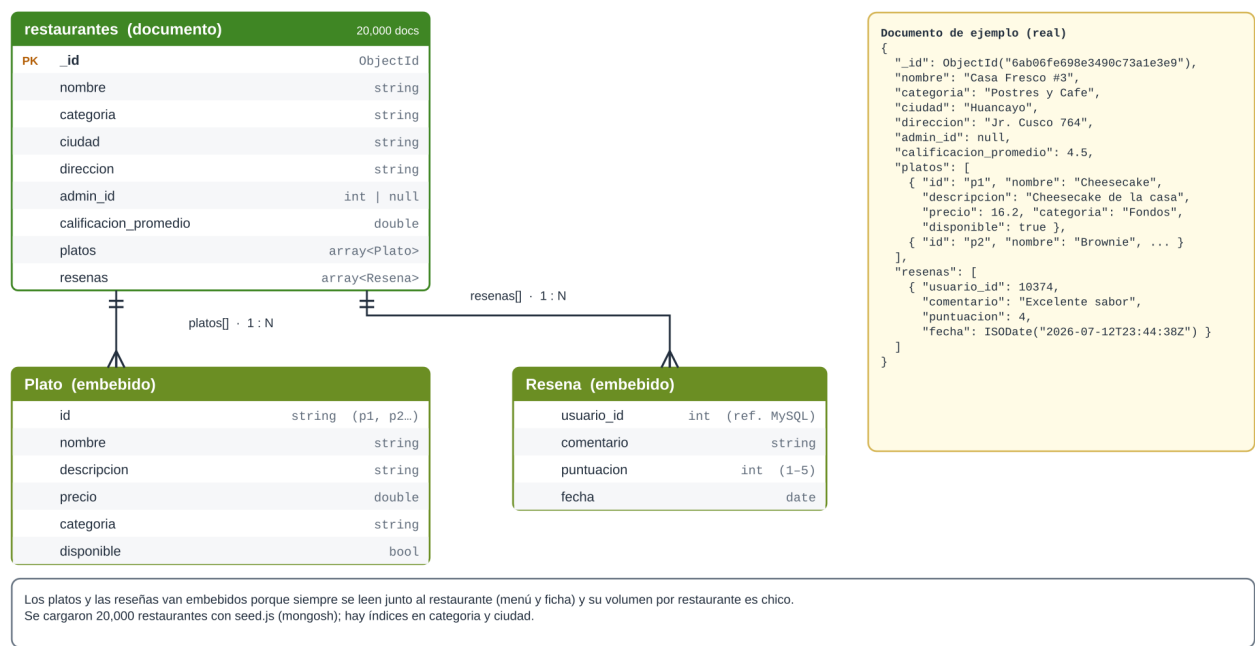


Figura 4. Estructura JSON de la colección restaurantes.

4.4 Carga masiva de datos

Los datos ficticios se insertaron una sola vez en cada base, con scripts incluidos en el repositorio. Los conteos son los del último snapshot cargado al data lake:

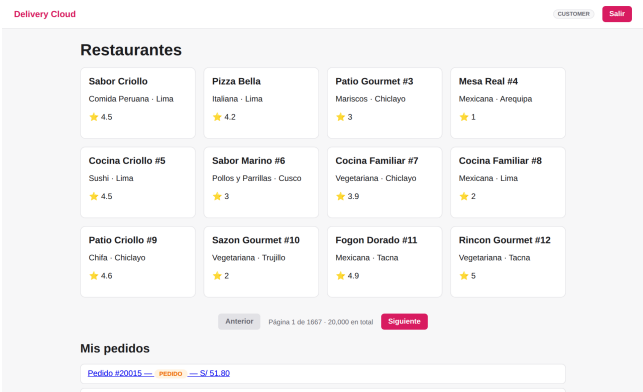
Base de datos	Tabla / colección	Script	Registros
MySQL (MS1)	usuarios	seed.py (Faker)	20,007
MySQL (MS1)	direcciones	seed.py	17,025
PostgreSQL (MS3)	orders	seed.js	20,009
PostgreSQL (MS3)	order_items	seed.js	26,566
MongoDB (MS2)	restaurantes	seed.js (mongosh)	20,000

5. Aplicación web (frontend)

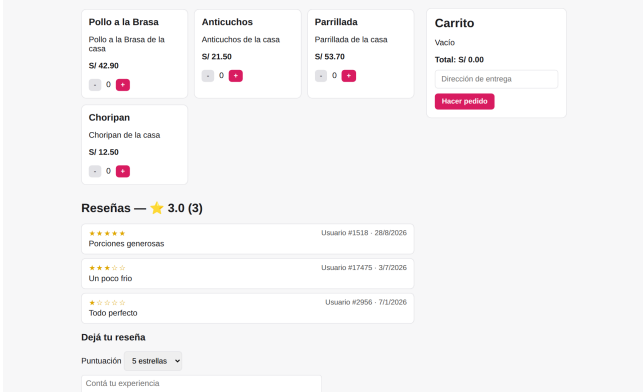
La aplicación es una SPA en **React con Vite**, con login y rutas protegidas por rol. Está desplegada en **AWS Amplify** (<https://main.duqtfdx5tvcw.amplifyapp.com>) y consume la API únicamente por el API Gateway con HTTPS.

Rol	Pantallas	Servicios que consume
Customer	Restaurantes (grilla paginada), menú con carrito y reseñas, mis pedidos y seguimiento del pedido	MS1, MS2, MS3, MS4
Admin	Panel de pedidos por estado (con los platos y cantidades de cada pedido), clientes, gestión del menú y analítica	MS2, MS3, MS4, MS5
Delivery	Pedidos disponibles, entregas en curso, marcar como entregado	MS3, MS4

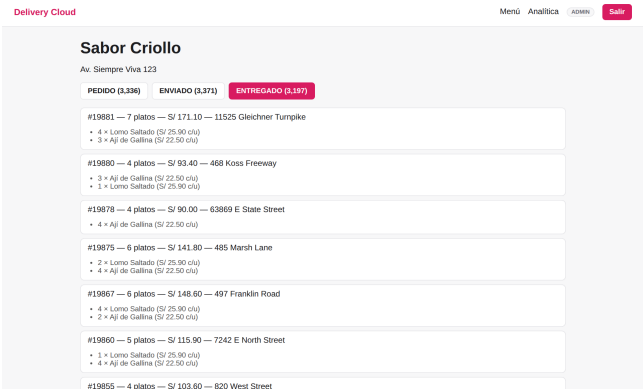
Pantallas de la aplicación



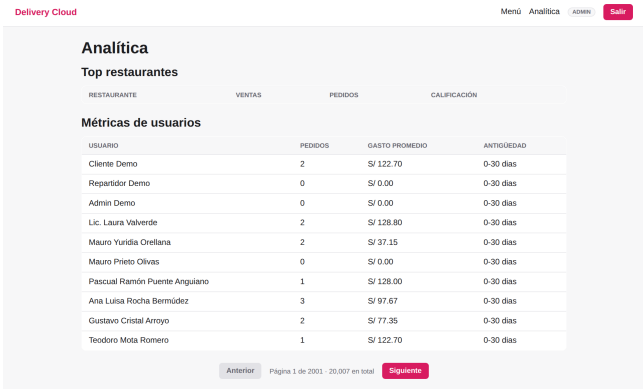
Customer: grilla de restaurantes paginada (20,000)



Customer: menú, reseñas y carrito



Admin: pedidos por estado con sus platos y cantidades



Admin: analítica con datos de Athena

Llamadas REST por microservicio

Servicio	Llamadas en el frontend	Ejemplos
MS1	5	login, registro, perfil, direcciones
MS2	9	listar restaurantes, ver menú, CRUD de platos, listar y crear reseñas
MS3	8	crear pedido, listar pedidos, cambiar de estado, tomar y entregar
MS4	4	tracking del pedido, dashboards de customer, delivery y admin
MS5	2	top de restaurantes, métricas de usuarios (consulta a Athena)

6. Despliegue en AWS

6.1 Recursos

Recurso	Detalle
PP-App-Tier	EC2 t2.micro, us-east-1a · NGINX + MS1...MS5 con docker compose · Elastic IP
PP-App-Tier-2	EC2 t2.micro, us-east-1b · idéntica a la anterior
PP-DB-Tier	EC2 t2.micro, us-east-1a · MySQL, PostgreSQL y MongoDB en contenedores con volúmenes persistentes
PP-Ingest-VM	EC2 t2.micro, us-east-1a · 3 contenedores de ingesta en Python
pp-alb-interno	Application Load Balancer interno, target group pp-app-tg (las dos VMs de App), health check GET /health cada 15 s
VPC Link + API Gateway	HTTP API con integración privada (VPC Link) hacia el listener del ALB; protocolo HTTPS hacia el cliente
AWS Amplify	Aplicación de una sola rama (main); despliegue del build de la SPA
S3 · Glue · Athena	Bucket pp-ubereats-datalake-...; base ubereats_datalake y crawler pp-crawler; workgroup pp-workgroup

6.2 Red y seguridad

Security group	Entrada permitida
pp-alb-sg (ALB)	TCP 80 solo desde pp-vpclink-sg
pp-app-tier-sg (VMs de App)	TCP 80 desde el ALB (también desde internet, ver sección 11); SSH restringido a una IP de administración
pp-db-tier-sg (VM de BD)	TCP 3306, 5432 y 27017 solo desde los SG de App e Ingesta; sin ningún puerto abierto a internet
pp-ingest-sg (Ingesta)	Ninguna entrada

- La administración de las VMs se hace con AWS Systems Manager (SSH sobre SSM), sin exponer el puerto 22 de la VM de BD.
- Las instancias usan el rol IAM del laboratorio (LabInstanceProfile): no hay claves de AWS en el código ni en los contenedores.
- El acceso a las bases de datos se comprobó desde internet: 22, 3306, 5432 y 27017 no responden.

6.3 Balanceo de carga

El ALB interno reparte las peticiones entre las dos VMs de App. Se comprobó con la cabecera *X-Served-By*: en 30 peticiones consecutivas al mismo endpoint, 14 las atendió app-1 y 13 app-2. Para verificar la tolerancia a fallos se detuvo NGINX en la segunda VM: el ALB la marcó como *unhealthy* en unos 30 s y las 30 peticiones siguientes las atendió app-1, todas con código 200. Al reiniciar NGINX, la VM volvió a rotación.

6.4 Cómo se actualiza

El código se despliega con `git pull` y `docker compose up --build` en cada VM (en segundo plano, con swap de 2.5 GB porque las VMs t2.micro tienen 1 GB de RAM), y después se reinicia NGINX, que resuelve las IPs de los contenedores al arrancar. El frontend se recompila con la URL del API Gateway y se sube a Amplify.

6.5 Documentación Swagger

Servicio	URL de Swagger UI (a través del API Gateway)
MS1	https://jt3z0elcud.execute-api.us-east-1.amazonaws.com/ms1/docs
MS2	https://jt3z0elcud.execute-api.us-east-1.amazonaws.com/ms2/swagger-ui.html
MS3	https://jt3z0elcud.execute-api.us-east-1.amazonaws.com/ms3/api-docs/
MS4	https://jt3z0elcud.execute-api.us-east-1.amazonaws.com/ms4/docs
MS5	https://jt3z0elcud.execute-api.us-east-1.amazonaws.com/ms5/docs

Detrás de NGINX y del API Gateway los servicios FastAPI viven bajo un prefijo (/ms1, /ms4, /ms5). NGINX envía la cabecera X-Forwarded-Prefix y un middleware la usa como root_path, de modo que Swagger UI carga el esquema y el botón «Try it out» arma bien las URLs.

7. Data Science: ingesta, catálogo y analítica

7.1 Ingesta y data lake

La **MV de ingesta** (PP-Ingest-VM) ejecuta **tres contenedores Docker en Python**, uno por microservicio, con estrategia *pull* del 100 % de las tablas. Cada uno genera archivos CSV (MySQL y PostgreSQL) o JSONL (MongoDB) y los sube al bucket de S3 en una carpeta por tabla y fecha (raw/mysql/usuarios/dt=AAAA-MM-DD/...). Antes de subir, cada contenedor elimina el snapshot anterior de esa tabla (*full refresh*), para que Athena no sume dos particiones.

Contenedor	Origen	Destino en S3	Formato
ingest-mysql	usuarios, direcciones (MS1)	raw/mysql/usuarios/ · raw/mysql/direcciones/	CSV
ingest-postgres	orders, order_items (MS3)	raw/postgres/orders/ · raw/postgres/order_items/	CSV
ingest-mongodb	restaurantes (MS2)	raw/mongodb/restaurantes/	JSONL

7.2 Catálogo de datos en AWS Glue

Un crawler (pp-crawler) recorre el bucket y crea una tabla por cada carpeta: **usuarios**, **direcciones**, **orders**, **order_items** y **mongodb** (el crawler nombró así a la colección de restaurantes). Los archivos no tienen claves foráneas; las relaciones del diagrama son lógicas y se resuelven con JOIN en Athena.

Diagrama Entidad-Relación del Catálogo de Datos — AWS Glue · base ubereats_datalake

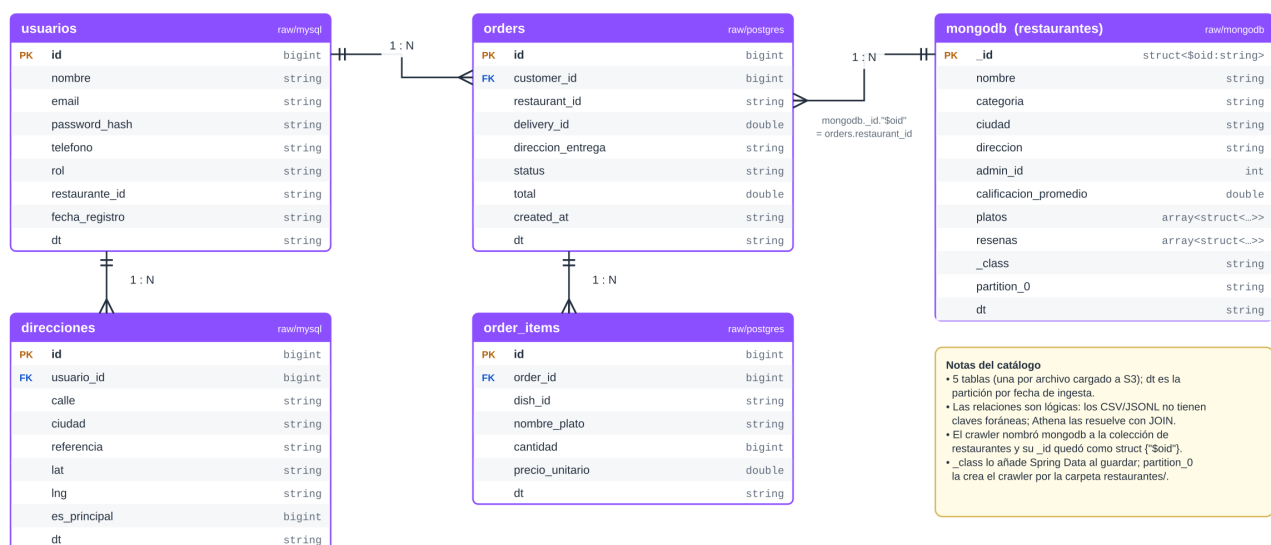


Figura 5. Diagrama E/R de las tablas del catálogo de Glue.

Hallazgos al catalogar: (1) el `_id` de MongoDB llegó como struct `{"_id"}` y se accede con `r._id["_id"]`; (2) `es_principal` (booleano en MySQL) llegó como bigint 0/1; (3) Spring Data agrega el campo `_class` a los documentos de MongoDB y el crawler agrega la partición `partition_0`; (4) el modelo de usuarios no tiene fecha de nacimiento, por lo que la vista de métricas usa la antigüedad de la cuenta en lugar de una edad inventada.

7.3 Consultas y vistas en Amazon Athena

Las consultas se ejecutaron en el workgroup pp-workgroup sobre la base ubereats_datalake. Cada resultado indica el **ID de ejecución de Athena**, que puede verificarse en el historial de consultas de la consola. Conteos del catálogo: usuarios = 20,007, orders = 20,009, order_items = 26,566, mongodb = 20,000.

Consulta 1 — Ventas por restaurante

Une orders, order_items y mongodb (3 tablas) para calcular pedidos y ventas totales por restaurante.

```
SELECT
  r.nombre AS restaurante,
  COUNT(DISTINCT o.id) AS num_pedidos,
  SUM(oi.cantidad * oi.precio_unitario) AS total_ventas
FROM orders o
JOIN order_items oi ON oi.order_id = o.id
JOIN mongodb r ON r._id.$oid = o.restaurant_id
GROUP BY r.nombre
ORDER BY total_ventas DESC;
```

restaurante	num_pedidos	total_ventas
Sabor Criollo	9,898	997,825.70
Pizza Bella	10,111	806,688.00

Athena: 6568912e-2abe-4bf4-8e09-ba40eaf7c46c · 2.5 s · 25866 KB escaneados · primeras 6 filas de 2 devueltas.

Consulta 2 — Clientes con mayor gasto

Une orders y usuarios para obtener el nombre real de los clientes que más gastan.

```
SELECT
  u.nombre AS cliente,
  u.email,
  COUNT(o.id) AS num_pedidos,
  SUM(o.total) AS gasto_total
FROM orders o
JOIN usuarios u ON u.id = o.customer_id
GROUP BY u.nombre, u.email
ORDER BY gasto_total DESC
LIMIT 50;
```

cliente	email	num_pedidos	gasto_total
Lucía Esteban Meraz Olivárez	user19878_nadiasolano@demo.com	9	870.80
Pedro Niño Cortés	user8656_villasenormaria-cristina@demo.com	6	731.30
José Carlos Gabriel Montez	user6641_jacobo81@demo.com	6	730.40
Eugenio Botello	user14013_cjaramillo@demo.com	8	700.40
Adán Sandoval Bueno	user18511_arellanodelia@demo.com	5	672.60
Gabriela Inés Vega Apodaca	user19231_rolandogarica@demo.com	6	671.80

Athena: 890bb508-1f76-4302-aa35-5b2d1be5a66f · 1.5 s · 5128 KB escaneados · primeras 6 filas de 10 devueltas.

Consulta 3 — Platos más vendidos

Une order_items, orders y mongodb (3 tablas) para obtener las unidades vendidas por plato y restaurante.

```
SELECT
  r.nombre AS restaurante,
  oi.nombre_plato,
  SUM(oi.cantidad) AS unidades_vendidas
FROM order_items oi
JOIN orders o ON o.id = oi.order_id
JOIN mongodb r ON r._id.$oid = o.restaurant_id
GROUP BY r.nombre, oi.nombre_plato
ORDER BY unidades_vendidas DESC
LIMIT 50;
```

restaurante	nombre_plato	unidades_vendidas
Pizza Bella	Pizza Margarita	25,209
Sabor Criollo	Ají de Gallina	20,640
Sabor Criollo	Lomo Saltado	20,593
Sabor Criollo	ala	1

Athena: 8b7213e4-57ce-4689-b348-92fc12f912b8 · 2.1 s · 25866 KB escaneados · primeras 6 filas de 4 devueltas.

Consulta 4 — Ciudad registrada vs. dirección de entrega

Une orders, usuarios y direcciones (LEFT JOIN) para comparar la ciudad del cliente con el destino del pedido.

```
SELECT
  u.nombre AS cliente,
  d.ciudad AS ciudad_registrada,
  o.direccion_entrega,
  COUNT(o.id) AS num_pedidos
FROM orders o
JOIN usuarios u ON u.id = o.customer_id
LEFT JOIN direcciones d ON d.usuario_id = u.id AND d.es_principal = 1
GROUP BY u.nombre, d.ciudad, o.direccion_entrega
ORDER BY num_pedidos DESC
LIMIT 50;
```

cliente	ciudad_registrada	direccion_entrega	num_pedidos
Cliente Demo	Lima	793 Gleason Cape	2
Cliente Demo	Lima	Av. Postman 1	2
Mario Matías Zamora	San Clara los altos	15778 Friesen Knolls	1
Jaime Marisol Soliz Pedroza		6417 Court Street	1
Mtro. Rafaél Henríquez	Vieja Omán	5437 Kuhlman Coves	1
Dr. Abigail Olvera		1236 Field Lane	1

Athena: 45f534a4-801f-4c86-a337-0863286fc75a · 1.8 s · 6802 KB escaneados · primeras 6 filas de 10 devueltas.

Vistas creadas

Se crearon dos vistas, consumidas por el microservicio MS5. Su resultado se muestra con un SELECT sobre cada vista.

v_resumen_ventas_restaurante

```
CREATE OR REPLACE VIEW v_resumen_ventas_restaurante AS
SELECT
  r._id."$oid" AS restaurante_id,
  r.nombre,
  COUNT(DISTINCT o.id) AS num_pedidos,
  SUM(o.cantidad * oi.precio_unitario) AS total_ventas,
  r.calificacion_promedio
FROM orders o
JOIN order_items oi ON oi.order_id = o.id
JOIN mongodb r ON r._id."$oid" = o.restaurant_id
GROUP BY r._id."$oid", r.nombre, r.calificacion_promedio;
```

restaurante_id	nombre	num_pedidos	total_ventas	calificacion_promedio
6aaf439f1bd3c2380450f532	Sabor Criollo	9,898	997,825.70	5.00
6aaf439f1bd3c2380450f533	Pizza Bella	10,111	806,688.00	4.20

CREATE VIEW: ac099df1-4664-43b3-98d6-750b933e0c1c · SELECT: 4aedf6fe-5bcc-43d7-86e5-106e51f5a3c8 · 2.2 s.

v_metricas_usuarios

```
CREATE OR REPLACE VIEW v_metricas_usuarios AS
SELECT
  u.id AS usuario_id,
  u.nombre,
  COUNT(o.id) AS num_pedidos,
  COALESCE(AVG(o.total), 0) AS gasto_promedio,
  CASE
    WHEN date_diff('day', date(substr(u.fecha_registro, 1, 10)), current_date) < 30 THEN '0-30 dias'
    WHEN date_diff('day', date(substr(u.fecha_registro, 1, 10)), current_date) < 90 THEN '31-90 dias'
    ELSE '90+ dias'
  END AS antiguedad_cuenta
FROM usuarios u
LEFT JOIN orders o ON o.customer_id = u.id
GROUP BY u.id, u.nombre, u.fecha_registro;
```

usuario_id	nombre	num_pedidos	gasto_promedio	antiguedad_cuenta
1	Cliente Demo	2	122.70	0-30 dias
2	Repartidor Demo	0	0.00	0-30 dias
3	Admin Demo	0	0.00	0-30 dias
4	Lic. Laura Valverde	2	128.80	0-30 dias
5	Mauro Yuridia Orellana	2	37.15	0-30 dias
6	Mauro Prieto Olivas	0	0.00	0-30 dias

CREATE VIEW: 13e4a748-b0ae-477a-ab76-4d5faa49c729 · SELECT: 8f42b3a6-307a-4c46-8e36-f795d3f5293a · 2.0 s.

8. Paginado y rendimiento

Con más de 20,000 registros por base, devolver listados completos era inviable (cientos de KB o MB por petición y miles de elementos en el DOM). Se añadió un **paginado opcional en la misma ruta**, que no rompe a los consumidores existentes:

- Sin *page* ni *page_size* la respuesta es la de siempre (un arreglo con todo).
- Con ellos la respuesta es {items, page, page_size, total, total_pages}. *page_size* vale 20 por defecto y se limita a 100; valores inválidos dan 400 (Express y Spring) o 422 (FastAPI).

- El total también viaja en la cabecera X-Total-Count (expuesta por CORS). Los dashboards de MS4 y la analítica de MS5 tienen su propio modo paginado; MS3 agrega /orders/summary para contar pedidos por estado sin traer las filas.

Petición	Sin paginar	Paginada
Dashboard admin (MS4)	4.2 MB · 0.84 s	4 KB · 60 ms
Usuarios (MS1)	6.6 MB · 2.3 s	página de 20: 14 ms
Restaurantes (MS2, 20,000)	21 MB · 1.0 s	página de 12: 12 KB · 14 ms
Pedidos de un restaurante (MS3)	miles de filas	página de 10: ≈ 10 ms
Métricas de usuarios (MS5)	consulta completa a Athena	página de 10: ≈ 3 a 4 s (latencia de Athena)

Mediciones tomadas en el entorno local con unos 40 mil pedidos (MS2 con sus 20,000 restaurantes); en AWS las respuestas son de un orden similar.

9. Pruebas y validación

Prueba	Alcance	Resultado
Colección Postman / Newman	72 requests sobre los 5 servicios: flujo completo, casos de error (400, 401, 422) y modos con y sin paginado	72 ejecutadas · 66 aserciones · 0 fallos, en local y a través del API Gateway
Navegador (Playwright)	Flujos por rol en Chromium contra la URL de Amplify: pestañas y paginado del admin, pool del delivery, grilla de restaurantes, pedido, tracking y reseñas	Todas las comprobaciones correctas y sin errores de consola
Balanceo y failover	30 peticiones repartidas; NGINX detenido en una VM	14 / 13 entre las dos VMs; con una caída, 30 de 30 respuestas 200
Aislamiento de la BD	Sondeo de puertos 22, 3306, 5432 y 27017 desde internet	Todos cerrados

10. Seguridad

- Autenticación con JWT (HS256, secreto de al menos 32 bytes) y contraseñas con bcrypt; autorización por rol en cada endpoint sensible.
- Red: BD sin puertos abiertos a internet, ALB interno, ingesta sin entradas y administración sin SSH público.
- Sin secretos en el repositorio: los archivos .env están en .gitignore y en AWS se usa el rol de la instancia.
- Observación: como la ingesta extrae el 100 % de las tablas, password_hash llega al data lake. En producción se excluiría o enmascararía esa columna antes de subirla.

11. Limitaciones y trabajo futuro

- **Puerto 80 de las VMs de App.** El tráfico de producción entra por API Gateway → ALB, pero las VMs aún aceptan el puerto 80 desde internet (por ejemplo por la IP elástica). Cerrarlo es una regla de red.
- **CI/CD del frontend.** Amplify se despliega subiendo el build; conectar la rama main de GitHub permitiría despliegues automáticos.
- **Recursos limitados.** Las VMs t2.micro (1 GB de RAM) funcionan con swap; en producción convendría t3.small o superior.
- **Reseñas.** MS2 toma el usuario_id del cuerpo de la petición y no del token; debería derivarse del JWT.
- **Data lake.** La ingesta es manual (bajo demanda); se podría programar con EventBridge y automatizar el crawler.

12. Lecciones aprendidas

- En AWS Academy Learner Lab las credenciales y las sesiones vencen, y al terminar la sesión las instancias quedan detenidas; los servicios se diseñaron para reiniciar solos (restart: unless-stopped).
- La plataforma limita a 9 el número de instancias de la cuenta y cuenta también las detenidas: las instancias nuevas se terminan a los segundos si se supera el límite.
- La AMI de Cloud9 trae Apache (puerto 80) y MySQL (3306) instalados: hay que deshabilitarlos antes de levantar NGINX y la base de datos.
- NGINX resuelve las IPs de los contenedores al arrancar: tras recrear un contenedor hay que reiniciarlo.
- Un crawler de Glue nombra las tablas según la estructura de S3 y puede convertir tipos (booleano a bigint, ObjectId a struct): conviene revisar el catálogo antes de escribir consultas.

Anexo A. Enlaces

Elemento	Enlace
Repositorio público (código fuente de los 5 microservicios, frontend, ingesta y diagramas)	https://github.com/miguelespinozaa-ship-it/ProeyctParcialProp
Aplicación web (AWS Amplify)	https://main.duqtfda5tvcw.amplifyapp.com
API pública (API Gateway, HTTPS)	https://jt3z0elcud.execute-api.us-east-1.amazonaws.com
Colección Postman	postman/delivery-cloud.postman_collection.json (en el repositorio)
Diagramas editables (draw.io)	infra/diagramas/proyecto-delivery-cloud.drawio (en el repositorio)

Anexo B. Cómo reproducir el proyecto

En local (cualquier persona que clone el repositorio):

```
git clone https://github.com/miguelespinozaa-ship-it/ProeyctParcialProp.git
cd ProeyctParcialProp
cp .env.example .env
docker compose -f docker-compose.dev.yml up -d --build
# servicios en http://localhost/ms1 ... /ms5
# Swagger: /ms1/docs /ms2/swagger-ui.html /ms3/api-docs /ms4/docs /ms5/docs
# frontend: cd frontend && npm install && npm run dev
# pruebas: npx newman run postman/delivery-cloud.postman_collection.json --env-var "base_url=http://localhost"
```

En AWS: las VMs de App usan docker-compose.app.yml, la VM de BD docker-compose.db.yml y la de ingesta data-science/docker-compose.ingest.yml. Los pasos de red (security groups, ALB, VPC Link y API Gateway) y los comandos de despliegue están descritos en el README del repositorio.